

AI Study Buddy - Project Analysis & Interview Preparation

SECTION 1 – Executive Summary

Project Overview

AI Study Buddy is an intelligent educational platform designed to enhance the learning experience by leveraging Generative AI. It allows users to upload their study materials (PDFs), from which it automatically extracts content, indexes it using a Retrieval-Augmented Generation (RAG) architecture, and provides interactive AI-driven tools. These tools include a conversational Q&A assistant grounded in the user's notes, automated quiz generation for self-assessment, and flashcard creation featuring spaced repetition for long-term retention.

Problem Solved

Traditional studying is often a passive and time-consuming process. Students struggle to identify key concepts in dense materials, formulate effective practice questions, and maintain consistent review schedules. AI Study Buddy solves this by automating the creation of active recall tools (quizzes and flashcards) and providing instant, personalized, and context-aware explanations (AI chat) based *only* on the student's actual course materials, thereby reducing hallucinations and increasing study efficiency.

Target Users

- **University/College Students:** Managing large volumes of reading materials and preparing for exams.
- **Lifelong Learners & Professionals:** Digesting complex documentation, research papers, or certification study guides.
- **Educators/Tutors:** Quickly generating quizzes and flashcards for their students based on lecture notes.

Features

1. **Document Management:** Upload PDFs (up to 10MB), background text extraction, and status tracking (processing, ready, error).
2. **Context-Aware AI Chat:** A conversational assistant that answers questions grounded strictly in the user's uploaded documents, complete with citations (e.g., source file and chunk reference).
3. **Automated Quiz Generation:** Generates 4-option multiple-choice quizzes with explanations, graded automatically, with attempt history and scoring.
4. **Spaced Repetition Flashcards:** Generates flashcard decks from notes. Includes a study session mode prioritizing cards based on mastery status (unseen, review, mastered).
5. **Dashboard & Analytics:** Real-time metrics tracking uploads, quiz scores, flashcard mastery, and study streaks.
6. **User Authentication & Profiles:** Secure JWT-based authentication, user preferences (theme, reminders), and profile management.

Functional & Non-Functional Requirements

Functional Requirements:

- System must accept PDF uploads, extract text, and chunk it.
- System must generate vector embeddings for chunks and store them.
- Users must be able to query their notes and receive AI responses with citations.
- Users must be able to generate, take, and review quizzes and flashcard decks.

- System must isolate data per tenant (user) to ensure privacy.

Non-Functional Requirements:

- **Performance:** Background processing for PDFs so the UI is not blocked.
- **Scalability:** Reference pattern in MongoDB (e.g., Messages in a separate collection from Chats) to bypass 16MB document limits.
- **Security:** Passwords hashed with bcrypt, JWTs stored in HttpOnly cookies, MongoDB injection sanitization, rate limiting on Auth and LLM routes.
- **Availability & Resilience:** Exponential backoff for LLM API calls, fallback from Gemini OCR to local pdf-parse.

Technology Stack

Frontend:

- **Core:** React (Vite), JavaScript (JSX).
- **Styling:** Tailwind CSS, Framer Motion (for animations).
- **State Management:** Zustand (modular slices for Auth, Chat, Quizzes, Flashcards).
- **Routing & Networking:** React Router DOM, Axios (with interceptors).

Backend:

- **Core:** Node.js, Express.js.
- **Database:** MongoDB (via Mongoose).
- **AI & LLMs:**
 - Google Gemini (@google/genai): Embeddings (gemini-embedding-2) and OCR/Text Extraction (gemini-3.5-flash).
 - Groq (groq-sdk): Lightning-fast LLM generation (llama-3.3-70b-versatile) for Chat, Quizzes, and Flashcards.
- **Vector Store:** Local JSON-based vector store using Cosine Similarity (a lightweight alternative to ChromaDB for easier local deployment, though the project contains chromadb in its

package.json).

- **Utilities:** Multer (PDF uploads), pdf-parse, Zod (validation), JWT, bcryptjs.

High-Level Architecture

1. **Client-Server Model:** A React SPA communicates with an Express REST API.
 2. **RAG Pipeline (Ingestion):**
 - PDF Upload -> Multer -> pdf-parse / Gemini OCR.
 - Text -> LangChain RecursiveCharacterTextSplitter -> Chunks.
 - Chunks -> Gemini Embeddings -> Local Vector Store.
 3. **RAG Pipeline (Retrieval):**
 - User Query -> Gemini Embeddings -> Cosine Similarity Search in Vector Store.
 - Top K Chunks + Query + Chat History -> Groq (Llama 3) -> Grounded Answer + Citations.
 4. **Data Layer:** MongoDB stores metadata (Users, Notes, Chats, Messages, Quizzes, Flashcards). Tenant isolation is enforced at the query level by requiring `ownerId` on all reads/writes.
-

SECTION 2 – Folder Structure

Root Directory

- `/backend`: The Express.js Node backend API.
- `/src`: The React frontend application.
- `/public`: Static frontend assets.
- `package.json`, `vite.config.js`, `tailwind.config.js`: Frontend configuration and dependencies.

Frontend (`/src`)

- **/components:** Reusable UI components.
 - `/ui`: Design system components (Button, Card, Input, ProgressBar, Badge, Skeleton).
- **/pages:** Route-level components mapping to specific views (Dashboard, Chat, Quizzes, QuizSession, Flashcards, FlashcardSession, Profile, Login, Register).
- **/store:** Zustand state management.
 - `index.js`: Combines all slices into a single hook.
 - `slices/`: Domain-specific state (`authSlice`, `chatSlice`, `quizSlice`, `flashcardSlice`, `uiSlice`).
- **/services:** API abstraction layer.
 - `_adapter.js`: The critical bridge that toggles between Mock Data and Axios based on the `VITE_USE MOCK` env variable.
 - `api.js`: Wrapper around the adapter.
- **/lib:** Utilities and configuration.
 - `axios.js`: Configured Axios instance with request/response interceptors for JWT injection and error handling.
- **App.jsx & main.jsx**: Application entry points and routing definitions.

Backend (/backend)

- **/src/server.js & /src/app.js**: Application bootstrapping, global middleware, error handlers, and route mounting.
- **/src/config**: Environment (`env.config.js`), Database (`db.config.js`), and Vector DB (`chroma.config.js`) configurations.
- **/src/routes**: Express routers grouped by domain (`auth`, `note`, `chat`, `quiz`, `flashcard`, `dashboard`).
- **/src/controllers**: Thin layer extracting HTTP request data (params, body), calling services, and returning standard JSON responses via `apiResponse.js`.
- **/src/services**: The core business logic layer.
 - `auth.service.js`: Registration, login, JWT generation.
 - `note.service.js`: Note CRUD and status tracking.
 - `chat.service.js`: Chat session management.
 - `quiz.service.js` & `flashcard.service.js`: Uses LLMs to generate structured assessments.

- `pdf.service.js`: PDF parsing with Gemini OCR fallback.
- `embeddings.service.js`: Text chunking, embedding generation, and cosine similarity search.
- `rag.service.js`: Orchestrates vector retrieval and LLM prompting for chat.
- `gemini.service.js` & `groq.service.js`: Wrappers around AI SDKs with exponential backoff and error handling.
- **/src/models**: Mongoose schemas defining the data layer (User, Note, Chat, Message, Quiz, FlashcardDeck).
- **/src/middlewares**:
 - `auth.middleware.js`: JWT verification.
 - `error.middleware.js`: Global error handling and normalization.
 - `rateLimiter.middleware.js`: API rate limiting.
 - `validate.middleware.js`: Zod schema validation for request payloads.
- **/src/utils**: Helpers (`AppError`, `catchAsync`, `logger`, `apiResponse`).
- **/data**: Local storage for `vectors.json`.
- **/uploads**: Temporary storage for uploaded PDFs.

Note: The architecture explicitly separates the Controller (HTTP layer) from the Service (Business Logic layer), making the codebase highly testable and maintainable.

SECTION 3 – Design Decisions & Trade-offs

1. Dual LLM Strategy (Gemini + Groq)

- **WHAT**: Using Google Gemini for OCR/Embeddings and Groq (Llama 3) for text generation.
- **WHY**: Groq offers exceptionally low latency (LPU hardware) making chat and quiz generation feel instantaneous. Gemini offers excellent multimodal OCR (reading PDFs) and high-quality

embeddings.

- **TRADE-OFFS:** Managing two API keys and rate limits.
- **INTERVIEW QUESTION:** *Why did you use two different AI providers instead of standardizing on OpenAI?*
 - *Answer:* "To optimize for latency and cost. Groq's LPU hardware makes generative tasks like chat almost instant, vastly improving UX. Gemini provides robust PDF parsing (OCR) and embeddings. It's a best-of-breed approach."

2. Controller-Service-Model Architecture (Backend)

- **WHAT:** Separating HTTP logic (Controllers) from Business Logic (Services) and Data Access (Models).
- **WHY:** Single Responsibility Principle. Controllers only parse requests and send responses. Services handle the heavy lifting (e.g., orchestrating RAG).
- **BENEFITS:** High testability (can test services without mocking HTTP), easier to migrate (e.g., adding a GraphQL layer or a CLI tool).

3. Asynchronous PDF Processing

- **WHAT:** When a user uploads a PDF, the HTTP response is sent immediately after saving the file, while text extraction, chunking, and embedding happen in the background (`processPdfInBackground`).
- **WHY:** Embedding large PDFs can take seconds to minutes. Blocking the HTTP response would cause timeouts and poor UX.
- **TRADE-OFFS:** Requires polling or a status UI on the frontend (implemented as 'processing' vs 'ready' states).
- **IMPROVEMENTS:** Implementing WebSockets or Server-Sent Events (SSE) for real-time status updates instead of polling, or using a dedicated message queue (Redis/BullMQ) for resilience.

4. Local Vector Store vs. Dedicated Vector DB (Chroma/Pinecone)

- **WHAT:** Embeddings are stored in a local `vectors.json` file and queried using a custom cosine similarity function.
- **WHY:** To eliminate the need for running a heavy Docker container (ChromaDB) during local development and testing, simplifying setup.
- **TRADE-OFFS:** Cannot scale beyond a single Node process. Memory usage grows linearly with the number of vectors.
- **IMPROVEMENTS:** In production, this should immediately be swapped for Pinecone, Weaviate, or ChromaDB.

5. Multi-Tenant Data Isolation

- **WHAT:** Every Mongoose query includes `ownerId` (e.g., `Note.find({ ownerId: req.user._id })`).
- **WHY:** Critical security measure in B2C SaaS to ensure users can never access each other's data (Insecure Direct Object Reference - IDOR prevention).

6. Zustand for State Management (Frontend)

- **WHAT:** Using Zustand instead of Redux or React Context.
 - **WHY:** Zustand provides a minimal boilerplate, global store that supports async actions cleanly and avoids unnecessary re-renders compared to Context API.
 - **HOW:** Slices (`createAuthSlice`, `createQuizSlice`) are merged into a single bound store in `store/index.js`.
-

SECTION 4 – Data Models & Database Schema

The database is NoSQL (MongoDB), designed for read efficiency and bounded document sizes.

1. User Model

- **Fields:** email, passwordHash, name, bio, preferences, stats.
- **Decisions:** Pre-save hook hashes the password using `bcryptjs`. Sensitive fields like `passwordHash` are excluded from default JSON serialization and queries (`select: false`).

2. Note Model

- **Fields:** ownerId (Ref: User), title, filePath, fileSizeKb, excerpt, status (processing/ready/error), extractedText.
- **Decisions:** `extractedText` is `select: false` by default to keep list queries lean. A compound index on `{ ownerId: 1, createdAt: -1 }` optimizes the dashboard feed.

3. Chat & Message Models (Reference Pattern)

- **Chat Fields:** ownerId, noteIds, title, lastActivityAt.
- **Message Fields:** chatId (Ref: Chat), ownerId, role (user/assistant), content, citations.
- **Decisions:** *Why aren't Messages embedded inside the Chat document?* MongoDB has a 16MB document limit. A long conversation history would exceed this limit and slow down queries. By using a separate `Message` collection with an index on `{ chatId: 1, timestamp: 1 }`, we can paginate histories efficiently.

4. Quiz Model (Embedded Pattern)

- **Fields:** ownerId, noteId, title, difficulty, questions (Array), attempts (Array).
- **Decisions:** *Why are Questions and Attempts embedded?* A quiz has a strict bound (max 20 questions) and a reasonable bound on attempts. They are always read and written together, making the Embedded Pattern highly efficient here.

5. FlashcardDeck Model

- **Fields:** `ownerId`, `noteId`, `title`, `cards` (Array).
- **Card Sub-Schema:** `front`, `back`, `tags`, `masteryStatus` (`unseen`, `review`, `mastered`), `lastReviewedAt`.
- **Decisions:** Uses spaced repetition logic on the backend (`getStudySession`) via an aggregation pipeline that sorts by `masteryStatus` and `lastReviewedAt` to fetch the next 20 cards.

SECTION 5 – API Design & RESTful Principles

Architecture

The backend exposes a RESTful JSON API.

- **Base Path:** `/api`
- **Domains:** `/auth`, `/notes`, `/chat`, `/quizzes`, `/flashcards`, `/dashboard`

REST Best Practices Followed

1. **Resource-Oriented URLs:** Routes use nouns, not verbs (e.g., `GET /api/notes`, not `/api/getNotes`).
2. **HTTP Methods:** Uses `GET` for retrieval, `POST` for creation, `PATCH` for partial updates (e.g., updating note title), and `DELETE` for removal.
3. **Standard Status Codes:**
 - `200 OK` (Success)
 - `201 Created` (Resource created)
 - `400 Bad Request` (Validation error)
 - `401 Unauthorized` (Missing/invalid JWT)
 - `403 Forbidden` (User doesn't own the resource)
 - `404 Not Found`
 - `429 Too Many Requests` (Rate limit exceeded)
 - `500 Internal Server Error`
4. **Standardized Response Envelope:** Every response uses a consistent wrapper via `apiResponse.js`.

```
{
  "status": "success",
  "data": { ... }
}
```

Or for errors:

```
{
  "status": "error",
  "error": { "code": "VALIDATION_FAILED", "message":
}
```

Request Validation

- **WHAT:** Using `zod` and `validate.middleware.js` to validate `req.body`, `req.query`, and `req.params`.
 - **WHY:** Prevents malformed data from reaching the controllers/services and crashing the app or causing DB inconsistencies.
-

SECTION 6 – Security Considerations

1. Authentication & Authorization

- **JWT (JSON Web Tokens):** Issued on login/register.
- **Storage:** The `refreshToken` is stored in an `HttpOnly`, `Secure`, `SameSite=strict` cookie. This prevents Cross-Site Scripting (XSS) attacks from stealing the token. The `accessToken` is sent in the response payload and kept in memory (Zustand store), which is wiped on page refresh (refresh token handles the rest).
- **Authorization:** `protect` middleware verifies the JWT and attaches `req.user`.

2. IDOR Protection (Insecure Direct Object Reference)

- Every single DB query includes `ownerId: req.user._id`. E.g., a user cannot access another user's note by simply guessing the

noteId in GET /api/notes/:id.

3. Injection Prevention

- **NoSQL Injection:** Handled by Mongoose schema casting and the `express-mongo-sanitize` middleware, which strips \$ and . characters from request payloads.
- **XSS Prevention:** Handled natively by React (which escapes strings in JSX).

4. Rate Limiting

- `express-rate-limit` is used to prevent brute-force and DDoS attacks.
- `globalLimiter`: Applied to all routes.
- `authLimiter`: Stricter limits (5 req / 15 min) for `/register` and `/login`.
- `llmLimiter`: Prevents API billing abuse for AI routes (10 req / 1 min).

5. File Upload Security

- **Multer Filter:** Only accepts `mimetype === 'application/pdf'`.
 - **Size Limit:** Max 10MB to prevent denial-of-service (DoS) via disk exhaustion.
-

SECTION 7 – RAG (Retrieval-Augmented Generation) Architecture Deep Dive

What is RAG?

RAG is a technique to improve LLM accuracy by providing it with external context. Instead of relying on the LLM's pre-trained memory, we search a database for relevant information and inject it into the prompt.

How it's implemented here:

Step 1: Ingestion (Background Job)

1. User uploads a PDF.
2. `pdf.service.js` reads it. Tries Gemini OCR first (for high accuracy on complex layouts). Falls back to local `pdf-parse` if Gemini fails.
3. `embeddings.service.js` uses LangChain's `RecursiveCharacterTextSplitter` to chunk the text (1000 characters, 200 overlap). Overlap ensures context isn't lost at the boundary of a chunk.
4. Gemini Embedding API converts chunks into high-dimensional vectors (arrays of floats).
5. Vectors are stored in a local JSON array with metadata (`noteId`, `ownerId`).

Step 2: Retrieval & Generation (`rag.service.js`)

1. User asks: "What is photosynthesis?"
2. The query is embedded using Gemini.
3. Cosine Similarity search compares the query vector against all vectors in the DB belonging to that user.
4. Top 5 chunks are retrieved.
5. A Prompt is constructed: "You are a tutor. Answer using ONLY the context below... [Chunks]".
6. Groq (Llama-3) generates the answer, citing the specific chunk metadata.

Interview Questions:

- *What happens if the PDF is 500 pages long?*
 - We chunk it. We only retrieve the top 5 most relevant chunks (approx 5000 characters) to fit into the LLM's

context window, keeping latency and cost low.

- *How do you prevent hallucinations?*
 - The system prompt strictly instructs the LLM: "Answer ONLY using the context. If not found, say 'I don't know'."
-

SECTION 8 – State Management (Frontend)

Zustand over Redux/Context

- **Why Zustand?** It is unopinionated, requires minimal boilerplate, and allows state to be accessed outside of React components (e.g., inside the Axios interceptor).
- **Architecture:** The store is split into functional slices (`authSlice`, `chatSlice`, `noteSlice`, etc.).
- **Data Fetching:** API calls are made directly inside the Zustand actions. The UI simply calls `store.fetchNotes()` and subscribes to `store.notes` and `store.isLoading`.

The Adapter Pattern (`_adapter.js`)

- The frontend features a highly decoupled service layer.
 - `src/services/_adapter.js` checks the `VITE_USE MOCK` environment variable.
 - If true, it returns mock data promises. If false, it calls the real Axios endpoints.
 - **Benefit:** Allows frontend developers to build and test the UI without spinning up the backend or using up API credits.
-

SECTION 9 – Error Handling & Logging

Global Error Handling (Backend)

- **AppError Class:** A custom Error class (`utils/AppError.js`) standardizes operational errors (status code, error code, message).
- **catchAsync:** A wrapper function for Express controllers that catches promise rejections and forwards them to `next()`, eliminating the need for `try/catch` blocks in every controller.
- **Global Error Middleware (`error.middleware.js`):** Intercepts all errors.
 - In Development: Returns full stack traces.
 - In Production: Hides server internals. Only sends messages for "operational" errors (AppError). Normalizes Mongoose errors (CastError, DuplicateKey) into operational errors.

Structured Logging

- `utils/logger.js` outputs JSON formatted logs.
 - **Redaction:** In production, it recursively scrubs sensitive keys (`password`, `token`, `apiKey`) from log metadata before writing to the console.
-

SECTION 10 – Performance & Scalability

Latency Optimization

1. **Groq LPU:** Using Groq for text generation reduces response times from ~5-10 seconds (standard GPU) to <1 second.
2. **Concurrent Aggregations:** In `dashboard.service.js`, metrics (notes, quizzes, flashcards, chats) are fetched simultaneously using `Promise.all()`.
3. **Compound Indexes:** MongoDB indexes (e.g., `{ ownerId: 1, createdAt: -1 }`) ensure fast sorting and querying for user feeds.

Scalability Limits

- **The Local Vector Store:** Writing to a single `vectors.json` file is a major bottleneck. It requires locking/sync issues at scale. This must be migrated to a dedicated vector database (Pinecone/Chroma) for production.
- **Background Jobs:** Currently, PDF processing blocks the Node.js event loop slightly (I/O heavy) and is tied to the web server process. At scale, this should be decoupled into a worker process (e.g., AWS SQS + Worker, or BullMQ/Redis).

SECTION 11 – Deployment & DevOps

Containerization (Docker)

- **WHAT:** The project includes a `docker-compose.yml` file.
- **WHY:** Docker ensures environment consistency across development, testing, and production (eliminating the "it works on my machine" problem).
- **Setup:** It spins up a `mongodb` container. Previously, it might have spun up a `chromadb` container, but currently, the system defaults to the local Node.js vector JSON fallback for even simpler local testing.

Production Deployment Strategy (Recommended)

- **Frontend:** Deployed to Vercel or Netlify. Vite's build process generates static HTML/JS/CSS files, which are distributed across a Global CDN for ultra-low latency.
 - **Backend:** Deployed to a PAAS like Render, Heroku, or AWS Elastic Beanstalk. Must support environment variables for API keys and DB URIs.
 - **Database:** MongoDB Atlas (fully managed cloud DB).
-

SECTION 12 – Testing Strategy

While testing code isn't heavily present in the MVP files, here is how you defend the strategy in an interview:

1. Unit Testing (Jest)

- Target: Services (e.g., `quiz.service.js`, `embeddings.service.js`).
- Mock external dependencies: Mock Mongoose models and API clients (GoogleGenAI, Groq) to test the business logic purely.

2. Integration Testing (Supertest)

- Target: API Endpoints (Controllers).
- Spin up an in-memory MongoDB instance (using `mongodb-memory-server`), hit the Express routes with `Supertest`, and verify status codes and JSON payloads.

3. End-to-End (E2E) Testing (Cypress / Playwright)

- Target: The React frontend.
 - Mock the API using Cypress intercepts (similar to what `_adapter.js` does) to test UI flows like logging in, generating a quiz, and submitting answers.
-

SECTION 13 – CI/CD Pipeline

To extend the project for an enterprise environment:

- **Continuous Integration (CI):** GitHub Actions triggers on every PR. It runs `npm run lint` (ESLint) and `npm run test` (Jest). If tests fail, the PR cannot be merged.
 - **Continuous Deployment (CD):** Upon merging to `main`, a GitHub Action builds the Docker image, pushes it to Docker Hub/AWS ECR, and triggers a redeployment on the hosting provider.
-

SECTION 14 – Third-Party Integrations

1. **Google Gemini API (@google/genai):** Used for generating vector embeddings and OCR parsing.
 - *Why:* Gemini has a highly capable multimodal engine that handles complex PDFs (tables, columns) better than traditional open-source OCR libraries.
 2. **Groq API (groq-sdk):** Used for executing the Llama-3 model.
 - *Why:* LPU (Language Processing Unit) architecture makes token generation incredibly fast, crucial for a snappy chat UI.
 3. **pdf-parse:** An open-source Node.js fallback.
 - *Why:* If the Gemini API is down or quota is exceeded, the app still functions (graceful degradation).
-

SECTION 15 – Edge Cases & Known Limitations

1. **Token Limits:** Large PDFs (100+ pages) might exceed rate limits or context windows. *Mitigation:* Chunking and only pulling the Top-K vectors using RAG.
 2. **Concurrency Issues:** Writing to the local `vectors.json` is not thread-safe. If two users upload PDFs simultaneously, the file could corrupt. *Fix:* Migrate to Pinecone or ChromaDB.
 3. **Stale JWTs:** If a user is deleted from the DB, their JWT remains valid until expiration. *Fix:* Add a `tokenVersion` to the User model, or check the DB on every request (which we do in `auth.middleware.js` via `User.findById`).
 4. **LLM Hallucinations:** Even with RAG, the LLM might hallucinate. *Fix:* Strict system prompting ("If you don't know, say I don't know").
-

SECTION 16 – Future Enhancements & Roadmap

1. **WebSocket Integration:** Replace polling with Socket.io for real-time PDF processing status updates.
 2. **Dedicated Vector Database:** Integrate Pinecone or Weaviate to replace the local JSON array for embeddings.
 3. **Advanced RAG:** Implement Hybrid Search (Keyword + Semantic) and Re-ranking (Cohere) to improve retrieval accuracy.
 4. **Rich Media Support:** Support image extraction from PDFs, or YouTube video link ingestion (via transcript fetching).
 5. **Social / Leaderboards:** Allow users to share flashcard decks and compete on quiz scores.
-

SECTION 17 – Common Interview Questions & Answers

Q: Why use a Reference pattern for Messages but an Embedded pattern for Quizzes? A: "MongoDB has a hard 16MB limit per document. A chat conversation can grow indefinitely, so embedding messages would eventually crash the app. Quizzes, however, have a strict bound (e.g., max 20 questions) and are always fetched together with their attempts, making embedding much faster as it avoids a `$lookup` (SQL Join)."

Q: How did you handle security in your API? A: "I implemented a layered approach. 1) JWTs for stateless auth, storing the refresh token in an `HttpOnly` cookie to prevent XSS. 2) `BCrypt` for password hashing. 3) `express-rate-limit` to prevent brute forcing. 4) Data isolation by ensuring every `Mongoose` query filters by `ownerId: req.user._id` to prevent IDOR."

Q: Walk me through what happens when a user asks a question in the chat. A: "The query goes to the backend where the Gemini API turns it into a vector. We calculate the cosine similarity against the user's stored document vectors to find the Top 5 most relevant text chunks. We inject these chunks into a system prompt alongside the user's query and send it to Groq's Llama-3 model. Groq generates the answer and cites the chunks, which we then store in MongoDB and return to the React

frontend."

SECTION 18 – Resume Bullet Points & Demo Script

Resume Bullet Points (Tailor to your role)

- **Full Stack / Backend:** "Architected an AI-powered educational platform using Node.js, Express, and MongoDB, implementing a Retrieval-Augmented Generation (RAG) pipeline to process PDFs and deliver context-aware LLM responses."
- **AI / Data:** "Engineered a custom vector search implementation using Gemini embeddings and Cosine Similarity, reducing dependency on external vector databases for local development."
- **Performance:** "Integrated Groq's LPU-accelerated Llama-3 model alongside Google Gemini, achieving sub-second generative response times for dynamic quizzes and chat queries."
- **Frontend:** "Developed a responsive React SPA using Vite, TailwindCSS, and Zustand, featuring interactive spaced-repetition flashcards and real-time dashboard analytics."

Demo Script (How to present this project)

1. **The Hook:** "Hi, this is AI Study Buddy. I built it because reading 50-page PDFs is passive and boring. I wanted a way to instantly chat with my textbooks and generate active-recall tools."
2. **The Upload:** "First, I'll upload a PDF on Biology. You'll notice it processes in the background so the UI doesn't freeze. Under the hood, we extract the text, chunk it, and embed it using Google Gemini."
3. **The Magic (Chat):** "Now I'll ask it a highly specific question from page 12. Because of the RAG architecture, the AI finds the exact chunk, answers the question, and provides a citation, proving it isn't hallucinating."

4. **The Value (Quiz/Cards):** "To test my knowledge, I hit 'Generate Quiz'. It hits the Groq API and instantly builds a 5-question test. Notice how it grades it and updates my dashboard stats in real-time."
5. **The Close:** "I built this using React, Node, and MongoDB, carefully handling things like rate-limiting, JWT security, and multi-tenant data isolation."